



Institute of Engineering and Technology (IET)

Major Project Synopsis(PR1107)

Real-Time Multiplayer Networking Under Unreliable Network Conditions

PREPARED BY:

Himanshi Sharma(2023Btech035)

Kopal Jain (2023Btech044)

Vinay Lunawat(2023Btech098)

FACULTY GUIDE

Devendra Bhavsar

Aug, 2026

Table Of Content

Table Of Content	2
1. Abstract	3
2. Introduction	4
3. Problem Statement	4
4. Objectives	4
5. Scope of Work	5
6. Methodology and Tech Stack	6
7. Fault Tolerance & Reliability	7
8. Expected Outcomes	7
9. References	8

1. Abstract

Real-time multiplayer games must remain responsive and consistent even when network conditions introduce delay, jitter, and message loss. In a conventional client-server design, the server acts as the authoritative source of game state, but network delay means that a player may see another player at a slightly older position than the position currently known by the server. This can make movement feel delayed and can cause a tag or collision to be rejected even when it appeared valid from the acting player's point of view.

This project proposes the design and development of a two-player real-time browser game that uses an authoritative Node.js server, client-side prediction, reconciliation, and server-side lag compensation. The clients communicate with the server through native WebSocket connections. The server maintains the authoritative simulation and a short history of recent player states, allowing it to evaluate tag interactions using the historical state corresponding to the time at which an input was generated.

The central experimental comparison is lagging compensation OFF versus lag compensation ON. Network conditions are degraded in a controlled and repeatable manner by introducing latency, jitter, and controlled message loss through a dedicated proxy. The evaluation focuses on hit-confirmation accuracy, correction magnitude, and a measurable fairness indicator defined as the difference in hit-success rate between a lower-latency player and a higher-latency player. A secondary target-side measure records the rate at which a tag is accepted after the target had already moved away from the tagged position on its own displayed timeline. The project therefore treats the game as an experimental platform rather than as the primary contribution. The expected result is a working multiplayer prototype together with reproducible measurements showing when lag compensation improves collision decisions and where the benefits begin to introduce trade-offs between players with different network conditions.

2. Introduction

Real-time multiplayer applications depend on a continuous exchange of state and player input between clients and a central server. A simple implementation can send player actions to the server, update the game state, and broadcast the resulting state back to the clients. However, this approach becomes visibly unresponsive as network delay increases because the local player must wait for a server response before seeing the result of an action.

To reduce this perceived delay, the proposed system uses client-side prediction for the local player. The browser immediately simulates the player's input while the same input is sent to the authoritative server. When the server response arrives, the client compares the authoritative state with its predicted state and performs reconciliation when a difference exists. This separates responsiveness from authority: the client can feel immediately, while the server remains the final source of truth.

3. Problem Statement

The project investigates how a server-authoritative two-player real-time game can maintain responsive movement and reliable tag/collision decisions when communication is affected by controlled latency, jitter, and message loss. The primary comparison is between lag compensation being disabled and enabled. The evaluation will measure hit-confirmation accuracy and the effect of network asymmetry on fairness. Fairness is operationalized as the absolute difference in hit-success rate between the lower-latency and higher-latency player, with the target-side post-dodge acceptance rate used as a secondary indicator of attacker-side advantage. The research question is: How does server-side lag compensation affect collision accuracy and player fairness in a real-time multiplayer game under controlled network degradation? Network conditions are tested separately where practical so that the effect of latency, jitter, and message loss can be observed without combining all failure modes into a single uncontrolled condition.

4. Objectives

- Design and implement a two-player real-time browser game with a server-authoritative simulation.
- Implement real-time communication using native browser WebSocket connections and a Node.js/TypeScript WebSocket server.
- Implement client-side prediction and reconciliation so that local movement remains responsive while authoritative server state is preserved.
- Maintain a short server-side history of player states and implement rewind-based lag compensation for tag/collision validation.
- Develop a controlled network-degradation proxy capable of injecting latency, jitter, and controlled message loss into client-server communication.
- Measure hit-confirmation accuracy, prediction correction magnitude, and fairness under different network conditions with lag compensation enabled and disabled, then analyze the collected data using Python, pandas, and matplotlib.

5. Scope of Work

The scope is deliberately limited to the networking and synchronization problems required to study the research question. The game itself will remain simple: two players connect to the same session, move around a small arena, and attempt to tag each other. The project does not aim to reproduce a commercial game, matchmaking service, account system, large-scale cloud architecture, or persistent game world.

- Client: TypeScript in the browser with HTML5 Canvas for rendering and the native WebSocket API for real-time communication.
- Server: Node.js with TypeScript, using the ws package and a custom fixed-timestep game loop. The server stores active game state in memory.
- Network test proxy: Rust as the preferred implementation for controlled delay, jitter, and message-loss injection. A Node.js proxy is retained as a fallback to reduce schedule risk.
- Logging and analysis: SQLite for persistent experiment records and Python with pandas/matplotlib for data processing, statistics, and graphs.
- Development and delivery: Vite for client development/builds and Git/GitHub for version control. Local execution is the primary test and demonstration environment; cloud hosting is optional only if an external-access demonstration is required.

6. Methodology

The methodology combines implementation of the networking stack with controlled experimentation. The system will first be built without lag compensation and validated under normal local conditions. Client-side prediction and reconciliation will then be introduced, followed by the server-side rewind mechanism used for tag validation. After functional validation, the proxy will be used to reproduce controlled adverse network conditions.

For each experiment, the same game scenario is executed repeatedly with a fixed set of network parameters. The primary independent variable is lag compensation state: OFF or ON. Network degradation is varied independently through added latency, jitter, and controlled message-loss rate. All other game parameters are kept constant for comparable runs.

- Authoritative simulation: every player input is processed by the server, which owns the official player positions, game tick/state, and tag result.
- Prediction and reconciliation: the local client immediately predicts its own movement, records unacknowledged inputs, and corrects the prediction when an authoritative server state arrives.
- State history and lag compensation: the server stores recent snapshots of player state. For a tag input, it estimates the relevant historical time and checks the collision against the corresponding past player position before restoring the current state.
- Experimental metrics: hit-confirmation accuracy is measured against the known simulation state at input time; fairness is measured as the absolute difference in hit-success rate between low-lag and high-lag players; correction magnitude quantifies prediction error; and target-side post-dodge acceptance rate captures cases where a tag is accepted after the target had already moved away on its displayed timeline.
- Data logging: each experiment records network configuration, lag-compensation mode, trial identifier, relevant timestamps, hit outcome, prediction correction, and player-specific results. SQLite is used for experiment persistence rather than for live game-state updates.
- Analysis: repeated trials are aggregated by network condition and compensation mode. Results are compared using tables and plots showing how collision accuracy and fairness change as network degradation increases.
- The system will be tested locally or on a controlled local network first, because reproducibility is more important to the experimental methodology than naturally occurring Internet conditions. A hosted server may be added only when needed for demonstration; it is not a core research dependency.

Tech Stack

- Client: TypeScript, HTML5 Canvas, native WebSocket
- Server: Node.js, TypeScript, ws
- Network proxy: Rust, with Node.js fallback
- Storage: SQLite
- Analysis: Python, pandas, matplotlib
- Build: Vite
- Version control: Git + GitHub

7. Fault Tolerance & Reliability

The reliability work focuses on maintaining a consistent simulation when network delivery is imperfect rather than on distributed infrastructure failures. The client should continue rendering predicted local movement while delayed updates are in transit, and the server must remain authoritative when messages arrive late or are missing.

- Delayed updates: the client continues to simulate locally and reconciles when newer authoritative state arrives instead of treating delayed state as a new input.
- Jitter: timestamped inputs and server state history are used so that variable message arrival time does not change the definition of the simulation timeline.
- Message loss: missing input/update messages are tolerated without transferring authority to the client; the server continues from its latest valid state.
- Disconnection handling: a dropped client connection is detected, and the match is ended or the disconnected player is removed without corrupting the remaining server state.
- Consistency validation: the server rejects impossible client-reported positions and bases tag resolution on authoritative simulation state rather than trusting client-supplied results.

8. Expected Outcomes

By the end of the project, the team expects to deliver both a functional prototype and an experimental evaluation of the chosen networking techniques.

- A working two-player browser game with authoritative server-side simulation and real-time WebSocket communication.
- A client-side prediction and reconciliation system that keeps local movement responsive while correcting prediction errors against authoritative state.
- A server-side lag-compensation mechanism that rewinds recent player state for tag/collision validation.
- A reproducible network-degradation test setup for controlled latency, jitter, and message-loss experiments.
- A SQLite-backed experimental dataset and Python-based analysis pipeline for generating graphs and comparing lag compensation OFF versus ON.
- Quantitative results describing changes in hit-confirmation accuracy, prediction correction, and fairness as network conditions worsen, including the trade-off between helping higher-latency players and increasing attacker-side advantage.

The intended contribution is not a feature-rich game but a measurable demonstration of how different synchronization techniques behave under controlled network conditions, supported by repeatable experiments and recorded evidence.

9. References

- [1] I. Fette and A. Melnikov, “The WebSocket Protocol,” RFC 6455, Internet Engineering Task Force, Dec. 2011. <https://www.rfc-editor.org/info/rfc6455/>
- [2] MDN Web Docs, “WebSocket API (WebSockets).” https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API
- [3] ws Project, “ws: a Node.js WebSocket library.” GitHub. <https://github.com/websockets/ws>
- [4] Y. W. Bernier, “Latency Compensating Methods in Client/Server In-game Protocol Design and Optimization,” Valve, 2001. Valve Developer Community.
- [5] C. Savery and T. C. Nicholas Graham, “Timelines: simplifying the programming of lag compensation for the next generation of networked games,” *Multimedia Systems*, vol. 19, pp. 271–287, 2013. DOI: 10.1007/s00530-012-0271-3.